SQL Order of Execution

The Logical Sequence of Query Processing

The "Golden Rule"

SQL is a **declarative** language. While we write queries starting with `SELECT`, the database engine processes the clauses in a specific logical order to build the result set efficiently. Understanding this order is essential for debugging and optimizing complex queries.

The Execution Flow (Step-by-Step)

Step	Clause	Logical Action
1	`FROM / JOIN`	Identifies the source tables and performs joins to create a base dataset.
2	`WHERE`	Filters the individual rows based on non-aggregate conditions.
3	`GROUP BY`	Groups the remaining rows into buckets based on unique values in specified columns.
4	`HAVING`	Filters the **groups** created in Step 3 (used for aggregate functions like `SUM`, `COUNT`).
5	`SELECT`	Determines which columns to return. Calculations, expressions, and **aliases** are processed here.
6	`DISTINCT`	Removes duplicate rows from the final column selection.
7	`ORDER BY`	Sorts the final result set. Since Step 5 is complete, you can use aliases here.
8	`LIMIT / OFFSET`	Restricts the number of rows returned to the user.

Visual Example

Consider the following query:

```sql
SELECT
    department_id,
    COUNT(*) AS employee_count     -- (5) Processed late: Alises born here
FROM employees                      -- (1) First: Get the data
WHERE salary > 50000                -- (2) Second: Filter raw rows
GROUP BY department_id              -- (3) Third: Group the data
HAVING COUNT(*) > 10                -- (4) Fourth: Filter the groups
ORDER BY employee_count DESC        -- (7) Seventh: Sort (Alias is safe here)
LIMIT 5;                            -- (8) Last: Truncate results
```

Critical "Gotchas" to Remember

- **The Alias Trap:** You cannot use a column alias defined in `SELECT` inside the `WHERE` clause.
  - *Reason:* The `WHERE` clause (Step 2) is evaluated before the `SELECT` clause (Step 5). The engine doesn't "know" the alias yet.
- **Aggregates & WHERE:** You cannot put a `SUM()` or `AVG()` in the `WHERE` clause.
  - *Reason:* `WHERE` filters data **before** grouping happens. Use `HAVING` to filter based on aggregated results.
- **Window Functions:** These are typically processed in the `SELECT` phase (Step 5). This is why you cannot use window function results in a `WHERE` clause without using a Subquery or CTE.

Created for professional SQL development. © 2024